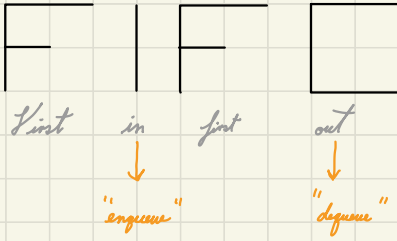


Quenes





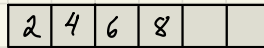
$T(n)$ costs

- $\mathcal{O}(1)$ • $q = \text{Queue}()$ # creating a queue
- $\mathcal{O}(1)$ • $\text{len}(q)$ # returns num of items in queue
- $\mathcal{O}(1)$ • $q.\text{is_empty}()$ # checks if $\text{len}(q) == 0$
- $\mathcal{O}_A(1)$ • $q.\text{enqueue}(\text{item})$ # add item at the end of queue
- $\mathcal{O}_A(1)$ • $q.\text{dequeue}()$ # removes item at front of queue
- $\mathcal{O}(1)$ • $q.\text{first}()$ # returns front item of queue

Implementation Models

- Traditional: "Everybody moves forward when dequeuing."

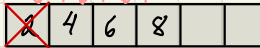
$q = \text{Queue}()$
 $q.\text{enqueue}(2)$
 $q.\text{enqueue}(4)$
 $q.\text{enqueue}(6)$
 $q.\text{enqueue}(8)$



front

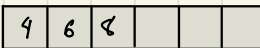
front

← shift to the left $\mathcal{O}(n)$



front

front

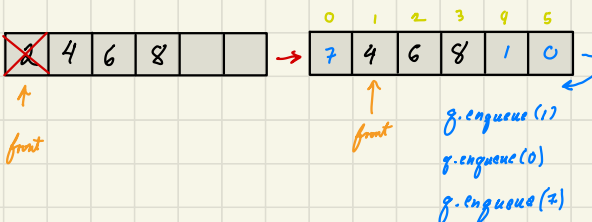


front

front

$q.\text{dequeue}()$ #2

- Circular: "Move the front one back if loop around when move line up"

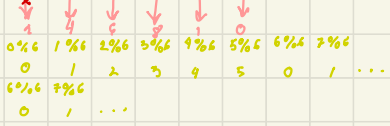
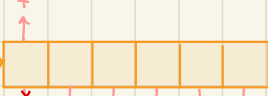


$$\text{index}_{\text{front}+1} = (\text{index}_{\text{front}} + 1) \% \text{capacity}$$

8

array
Queue

Array Queue
data_array
first
num_of_ele



first
↓

